Introduction to Greedy Algorithm

Understanding Greedy Algorithm

A Greedy algorithm is used to solve problems in which the best choice is made at each step, and it finds a solution in a minimal step. This approach assumes that choosing a local optimum at each stage will lead to the determination of a global optimum. It's like making the most beneficial decision at every crossroad, hoping it leads to the ultimate destination efficiently.

Purpose of Greedy Algorithm

The main goal of a greedy algorithm is to solve complex problems by breaking them down into simpler subproblems, solving each one optimally to find a solution that is as close as possible to the overall optimum. It's particularly useful in scenarios where the problem exhibits the properties of greedy choice and optimal substructure.

Properties for Greedy Algorithm Suitability

1. **Greedy Choice Property**: The local optimal choices lead to a global optimum, meaning the best solution in each small step leads to the best overall solution.
2. **Optimal Substructure**: A problem has an optimal substructure if an optimal solution to the entire problem contains the optimal solutions to its subproblems.

Characteristics of Greedy Method

1. **Local Optimization**: At every step, the algorithm makes a choice that seems the best at that moment, aiming for local optimality.
2. **Irrevocability:** Once a choice is made, it cannot be undone or revisited. This is a key characteristic that differentiates greedy methods from dynamic programming, where decisions can be re-evaluated.

Components of Greedy Algorithm

1. **Candidate Set:** The set of choices available at each step.
2. **Selection Function**: This function helps in choosing the most promising candidate to be added to the current solution.
3. **Feasibility Function:** It checks if a candidate can be used to contribute to a solution without violating problem constraints.
4. **Objective Function**: This evaluates the value or quality of the solution at each step.
5. **Solution Function:** It determines if a complete solution has been reached.

Simplified Greedy Algorithm Process

1. **Start with an Empty Solution Set:** The algorithm begins with no items in the solution set.
2. **Iterative Item Selection:** In each step, choose the most suitable item based on the current goal.
3. **Add Item to Solution Set:** The selected item is added to the solution set.
4. **Feasibility Check**: Determine if the solution set with the new item still meets the problem's constraints.
5. **Accept or Reject the Item**:
    - **If Feasible**: Keep the item in the solution set.
    - **If Not Feasible:** Remove and permanently discard the item.
6. **Repeat Until Complete**: Continue this process until a full solution is formed or no feasible solution can be found.
7. **Assess Final Solution:** Evaluate the completed solution set against the problem's objectives.

Let's understand the greedy algorithm via the example below.

Problem Statement (Boats to Save People)

We are given an array `people` where each element `people[i]` represents the weight of the `i`-th person. There is also a weight `limit` for each boat. Each boat can carry at most two people at a time, but the combined weight of these two people must not exceed `limit`. The objective is to determine the minimum number of boats required to carry all the people.

Example

Input: `people = [10, 55, 70, 20, 90, 85]`, `limit = 100`
Output: `4`
Explanation: One way to transport all people using 4 boats is as follows:

- Boat 1: Carry people with weights 10 and 90 (total weight = 100).
- Boat 2: Carry a person with weight 85 (total weight = 85).
- Boat 3: Carry people with weights 20 and 70 (total weight = 90).
- Boat 4: Carry people with weights 55 (total weight = 55).

Algorithm

1. **Sort the Array**: Sort the `people` array in ascending order.

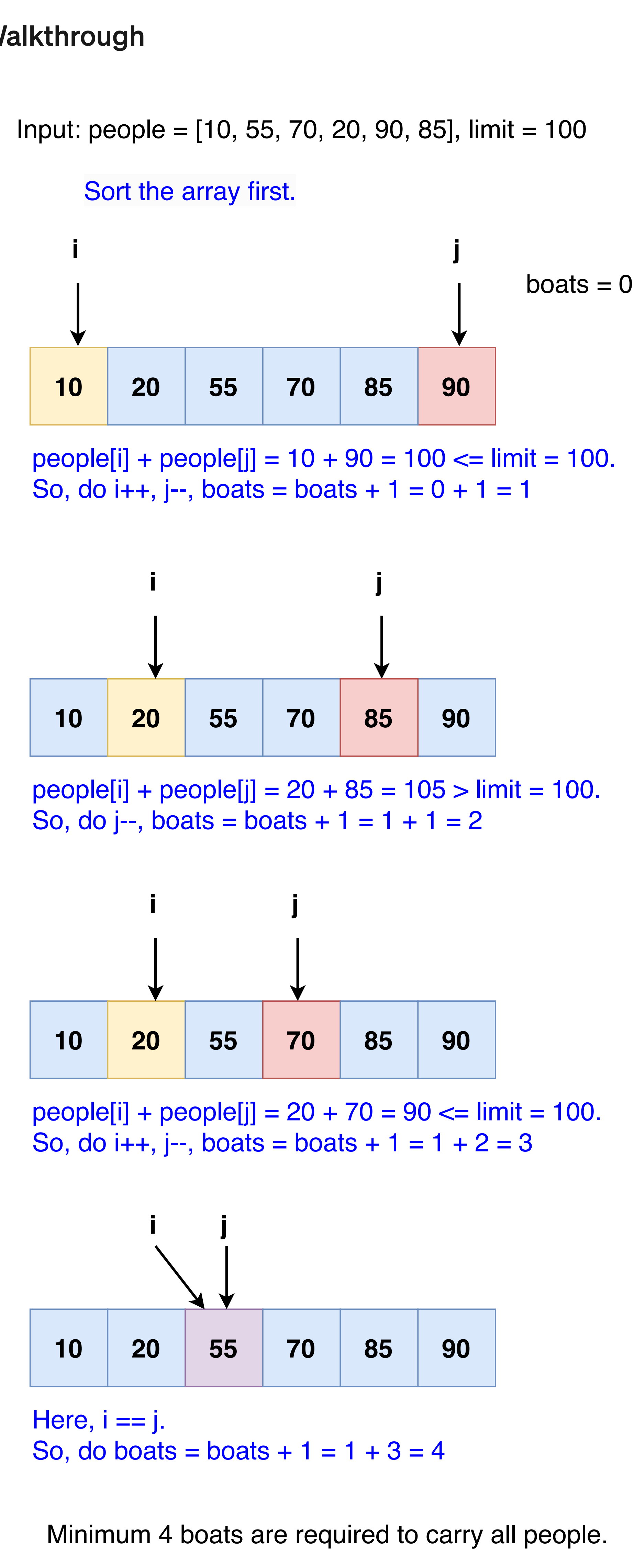
```
Arrays.sort(people);
```

2. **Initialize Two Pointers**: Set two pointers, `i` at the start (lightest person) and `j` at the end (heaviest person) of the array.
3. **Select Optimal Pairs**: Iterate through the array and pair the lightest person with the heaviest person if their combined weight is within the limit.

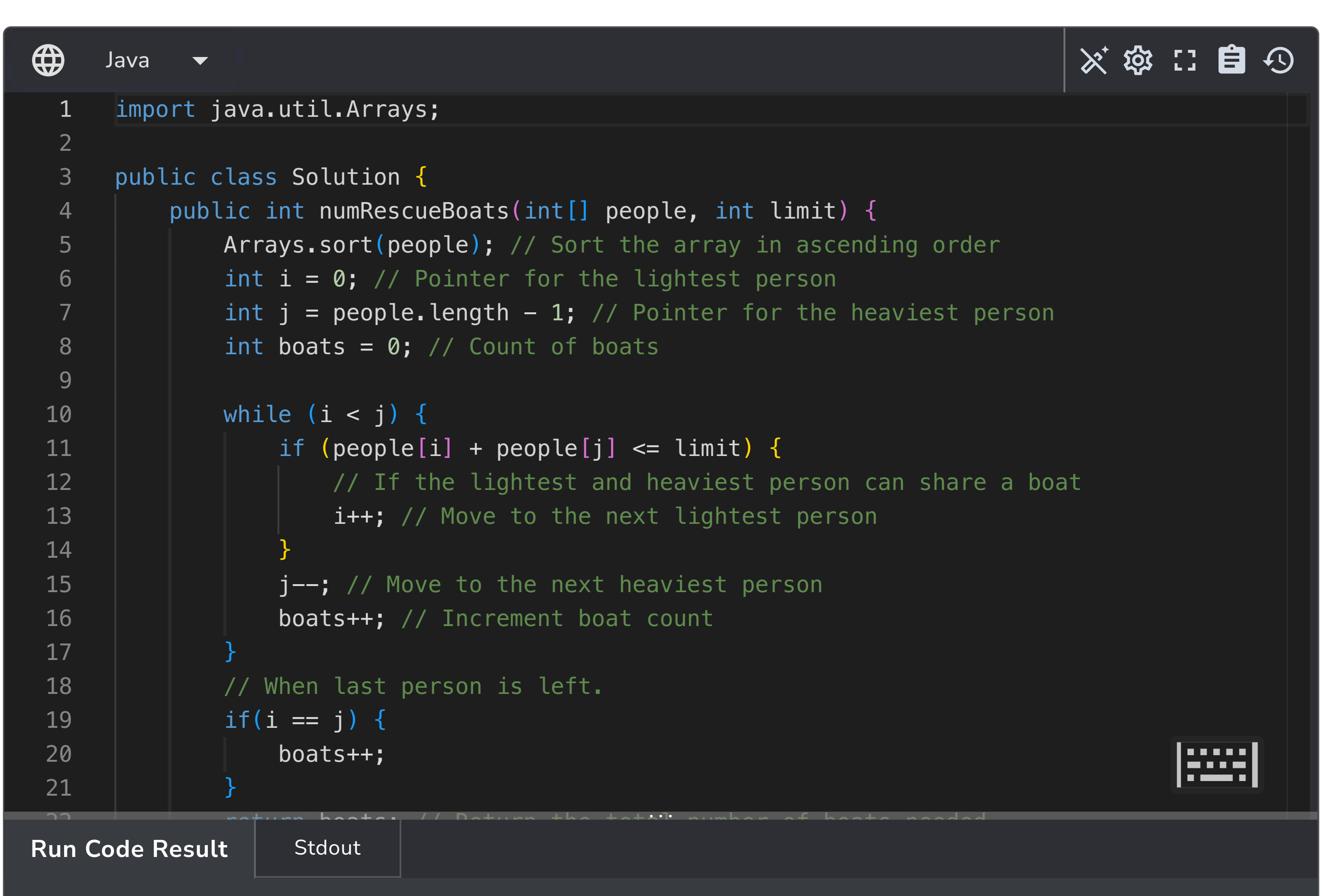
```
if (people[i] + people[j] ≤ limit) {
    i++; // Move to the next lightest person
}
```

4. **Count Boats**: For each iteration, increment the boat count. Move the `j` pointer to the left (next heaviest person).
5. **Repeat Until All Are Accounted For**: Continue until all people are accounted for (i.e., until `i` < `j`).
6. If i == j, increment the boat count by 1 to arrange boat for last remaining person.
7. **Return the Boat Count**: The total count of boats is the minimum number required.

Algorithm Walkthrough


Input: people = [10, 55, 70, 20, 90, 85], limit = 100
Sort the array first.
i
j
boats = 0
10
20
55
70
85
90
people[i] + people[j] = 10 + 90 = 100 <= limit = 100.
So, do i++, j--, boats = boats + 1 = 0 + 1 = 1
people[i] + people[j] = 20 + 85 = 105 > limit = 100.
So, do j--, boats = boats + 1 = 1 + 1 = 2
people[i] + people[j] = 20 + 70 = 90 <= limit = 100.
So, do i++, j--, boats = boats + 1 = 1 + 2 = 3
Here, i == j.
So, do boats = boats + 1 = 1 + 3 = 4
Minimum 4 boats are required to carry all people.

Code



```
import java.util.Arrays;

public class Solution {
    public int numRescueBoats(int[] people, int limit) {
        Arrays.sort(people); // Sort the array in ascending order
        int i = 0; // Pointer for the lightest person
        int j = people.length - 1; // Pointer for the heaviest person
        int boats = 0; // Count of boats

        while (i < j) {
            if (people[i] + people[j] <= limit) {
                // If the lightest and heaviest person can share a boat
                i++; // Move to the next lightest person
            }
            j--; // Move to the next heaviest person
            boats++; // Increment boat count
        }
        // When last person is left.
        if(i == j) {
            boats++;
        }
```

You must run your code first

In the above problem, the greedy approach is applied through the strategy of **pairing the lightest and heaviest individuals** to optimize boat usage. After sorting the people by weight, the algorithm iteratively pairs the lightest person (at the start of the array) with the heaviest (at the end), maximizing the use of each boat's capacity. This method ensures that each boat carries as much weight as possible without exceeding the limit, effectively reducing the total number of boats needed.

The essence of the greedy method here is making the **most efficient pairing choice at each step**, aiming for an overall optimal solution - the minimum number of boats to transport everyone.

Pros of Greedy Approach

- **Efficiency in Time and Space**: Greedy algorithms often have lower time and space complexities, making them fast and memory-efficient.
- **Ease of Implementation**: They are generally simpler and more straightforward to code than other complex algorithms.
- **Optimal for Certain Problems**: In problems with certain structures, like those with greedy-choice property and optimal substructure, greedy algorithms guarantee an optimal solution.
- **Useful for Approximations**: When an exact solution is not feasible, greedy algorithms can provide a close approximation quickly.

Cons of Greedy Approach with Example

- **Not Always Optimal**: Greedy algorithms do not always yield the global optimum solution, especially in problems lacking a greedy-choice property.
- **Shortsighted Approach**: They make decisions based only on current information, without considering the overall problem.
- **Example:** Consider the below tree where each path has a certain weight, and we need to find the path with the highest weight.

```
       2
     /   \
    1     3
   / \     \
  20   8     7
```

In the given tree, a greedy approach, which selects the path with the highest immediate weight at each step, would choose the path **2 → 3 → 7**, resulting in a total weight of 12.

However, this is not the optimal path. The path **2 → 1 → 20**, although starting with a lower weight at the first decision point, leads to a higher total weight of 23. This example demonstrates how the greedy approach can fail to find the best solution, as it overlooks the long-term benefits of initially less attractive choices.

Standard Greedy Algorithms

- **Kruskal's Minimum Spanning Tree Algorithm**: Builds a minimum spanning tree by adding the shortest edge that doesn't form a cycle.
- **Prim's Minimum Spanning Tree Algorithm**: Grows a minimum spanning tree from a starting vertex by adding the nearest vertex.
- **Dijkstra's Shortest Path Algorithm**: Finds the shortest path from a single source to all other vertices in a weighted graph.
- **Huffman Coding**: Used for data compression, it builds a binary tree with minimum weighted path lengths for given characters.
- **Fractional Knapsack Problem**: Maximizes the total value of items in a knapsack without exceeding its capacity.
- **Activity Selection Problem**: Select the maximum number of activities that don't overlap in time.
- **Greedy Best-First Search**: Used in AI for pathfinding and graph traversal, prioritizing paths that seem closest to the goal.

Let's start solving the problem which are based on the greedy algorithm.

← Previous
Solution: Problem Challenge 1: K Pairs With Largest Sum

Next →
Valid Palindrome II (Easy)

☐ Mark as Completed